Human Face Recognition using Deep Learning

This notebook trains a celebrity face recognition system using FaceNet embeddings, MTCNN face detection, and a TensorFlow/Keras classifier. It is written to run in Google Colab from top to bottom.

```
# Cell 1: Upload kaggle.json, configure Kaggle, download celebrity dataset, unzip, and organize folders
import os
import shutil
import zipfile
import subprocess
from pathlib import Path

IN_COLAB = False
try:
    import google.colab
    IN_COLAB = True
except Exception:
    IN_COLAB = False

BASE_DIR = Path('/content/Human-Face-Recognition') if IN_COLAB else Path.cwd() / 'Human-Face-Recognition'
DATASET_DIR = BASE_DIR / 'dataset'
RAW_DIR = DATASET_DIR / 'raw'
STRUCTURED_DIR = DATASET_DIR / 'structured'
MODELS_DIR = BASE_DIR / 'models'

for folder in [BASE_DIR, DATASET_DIR, RAW_DIR, STRUCTURED_DIR, MODELS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

DATASET_SLUG = 'bhaveshmittal/celebrity-face-recognition-dataset'
ZIP_PATH = DATASET_DIR / 'celebrity-face-recognition-dataset.zip'

if not IN_COLAB:
    raise EnvironmentError('This notebook is intended for Google Colab. Open it in Colab and run again.')

from google.colab import files
print('Upload kaggle.json to continue...')
uploaded = files.upload()
if 'kaggle.json' not in uploaded:
    raise FileNotFoundError('kaggle.json not uploaded. Rerun this cell and upload the file.')

os.makedirs('/root/.kaggle', exist_ok=True)
with open('/root/.kaggle/kaggle.json', 'wb') as f:
    f.write(uploaded['kaggle.json'])
os.chmod('/root/.kaggle/kaggle.json', 0o600)

subprocess.run(['pip', 'install', '-q', 'kaggle'], check=True)
if not ZIP_PATH.exists():
    subprocess.run(['kaggle', 'datasets', 'download', '-d', DATASET_SLUG, '-p', str(DATASET_DIR)], check=True)

with zipfile.ZipFile(ZIP_PATH, 'r') as zip_ref:
    zip_ref.extractall(RAW_DIR)

image_exts = {'.jpg', '.jpeg', '.png', '.webp'}
for img_path in RAW_DIR.rglob('*'):
    if img_path.is_file() and img_path.suffix.lower() in image_exts:
        class_name = img_path.parent.name.strip().replace('  ', ' ')
        target_dir = STRUCTURED_DIR / class_name
        target_dir.mkdir(parents=True, exist_ok=True)
        target_path = target_dir / img_path.name
        if not target_path.exists():
            shutil.copy2(img_path, target_path)

class_dirs = [p for p in STRUCTURED_DIR.iterdir() if p.is_dir()]
print('Structured dataset ready at:', STRUCTURED_DIR)
print('Total celebrity classes found:', len(class_dirs))
print('Sample classes:', [p.name for p in class_dirs[:10]])
```

```
Upload kaggle.json to continue...
Choose Files kaggle.json
kaggle.json(application/json) - 75 bytes, last modified: 4/26/2026 - 100% done
Saving kaggle.json to kaggle (2).json
Warning: You uploaded 'kaggle (2).json'. The notebook expects 'kaggle.json'. Using 'kaggle (2).json' to configure Kaggle.
Structured dataset ready at: /content/Human-Face-Recognition/dataset/structured
Total celebrity classes found: 17
Sample classes: ['Kate Winslet', 'Nicole Kidman', 'Jennifer Lawrence', 'Natalie Portman', 'Denzel Washington', 'Tom Cruise', 'Me
```

```
# Cell 2: Install dependencies
import sys
import subprocess

packages = [
    'numpy', 'pandas', 'matplotlib', 'seaborn', 'opencv-python', 'tensorflow',
    'scikit-learn', 'joblib', 'tqdm', 'mtcnn', 'keras-facenet', 'deepface', 'requests'
]
subprocess.run([sys.executable, '-m', 'pip', 'install', '-q'] + packages, check=True)
print('Dependencies installed.')
```

```
Dependencies installed.
```

```
# Cell 3: Imports and configuration
import io
import json
import random
import warnings
from dataclasses import dataclass
from pathlib import Path

import cv2
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests
import seaborn as sns
import tensorflow as tf
from IPython.display import display
from keras import Sequential
from keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from keras.layers import BatchNormalization, Dense, Dropout, Input
from keras.models import load_model
from keras_facenet import FaceNet
from mtcnn import MTCNN
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score
from sklearn.metrics.pairwise import cosine_distances
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tqdm.auto import tqdm

warnings.filterwarnings('ignore')
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

gpus = tf.config.list_physical_devices('GPU')
print('TensorFlow version:', tf.__version__)
print('GPU available:', bool(gpus))
if gpus:
    print(gpus)

@dataclass
class Config:
    image_size: tuple = (160, 160)
    min_images_per_class: int = 20
    max_classes: int = 17
    test_size: float = 0.15
    val_size: float = 0.15
    batch_size: int = 32
    epochs: int = 10
    unknown_distance_threshold: float = 0.40
```

```
    classifier_confidence_threshold: float = 0.60

CFG = Config()
detector = MTCNN()
embedder = FaceNet()
```

```
TensorFlow version: 2.19.0
GPU available: False
```

```
# Cell 4: Dataset loading and EDA helpers
def build_dataframe(dataset_root, min_images_per_class=20, max_classes=17):
    rows = []
    class_counts = []
    image_exts = {'.jpg', '.jpeg', '.png', '.webp'}
    for class_dir in sorted(dataset_root.iterdir()):
        if class_dir.is_dir():
            images = [p for p in class_dir.iterdir() if p.suffix.lower() in image_exts]
            class_counts.append((class_dir.name, len(images)))
    class_counts = [x for x in class_counts if x[1] >= min_images_per_class]
    class_counts = sorted(class_counts, key=lambda x: x[1], reverse=True)[:max_classes]
    keep_classes = {name for name, _ in class_counts}
    for class_dir in sorted(dataset_root.iterdir()):
        if class_dir.is_dir() and class_dir.name in keep_classes:
            for img_path in class_dir.iterdir():
                if img_path.suffix.lower() in image_exts:
                    rows.append({'label': class_dir.name, 'image_path': str(img_path)})
    df = pd.DataFrame(rows)
    if df.empty:
        raise ValueError('No usable data found. Reduce min_images_per_class or check extraction.')
    return df

def plot_distribution(df):
    plt.figure(figsize=(14, 5))
    sns.countplot(data=df, x='label', order=df['label'].value_counts().index)
    plt.xticks(rotation=45, ha='right')
    plt.title('Celebrity image count by class')
    plt.tight_layout()
    plt.show()

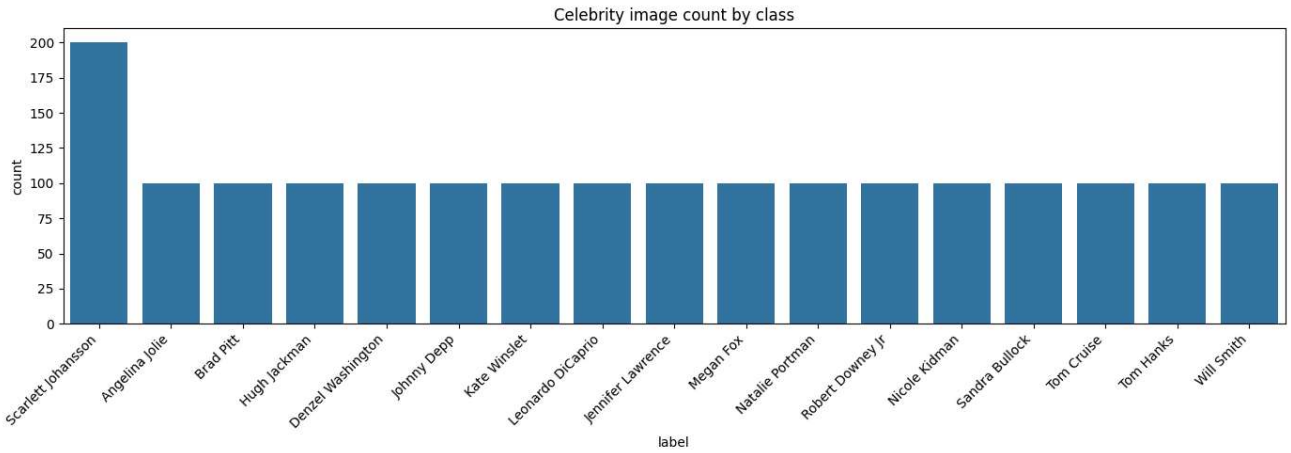
def show_samples(df, n=12):
    sample_df = df.sample(min(n, len(df)), random_state=SEED).reset_index(drop=True)
    fig, axes = plt.subplots(3, 4, figsize=(12, 9))
    for ax, (_, row) in zip(axes.flat, sample_df.iterrows()):
        image = cv2.cvtColor(cv2.imread(row['image_path']), cv2.COLOR_BGR2RGB)
        ax.imshow(image)
        ax.set_title(row['label'])
        ax.axis('off')
    plt.tight_layout()
    plt.show()

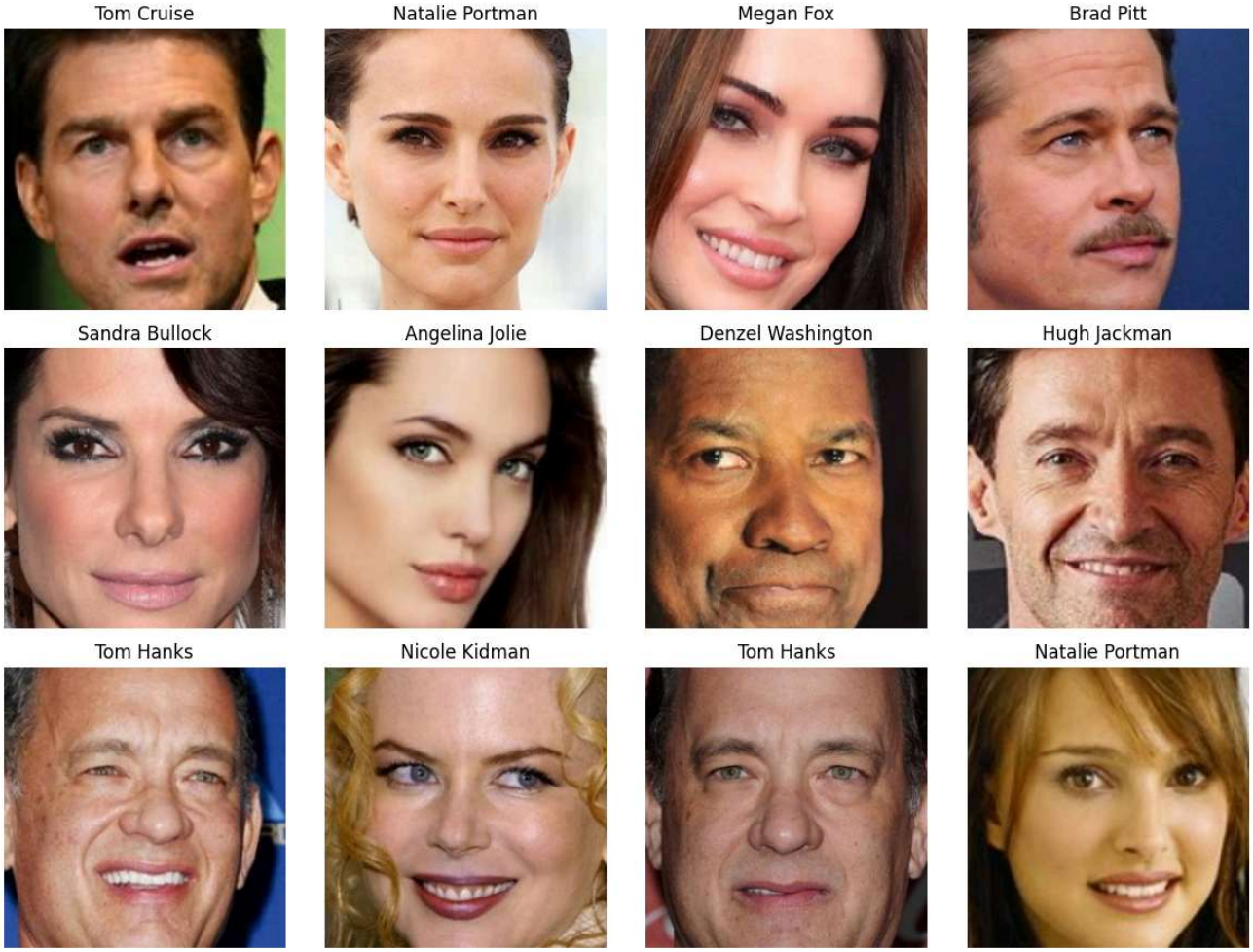
df = build_dataframe(STRUCTURED_DIR, CFG.min_images_per_class, CFG.max_classes)
print('Total images:', len(df))
print('Classes:', df['label'].nunique())
display(df.head())
```

```
Total images: 1800
Classes: 17
```

	label	image_path
0	Angelina Jolie	/content/Human-Face-Recognition/dataset/struct...
1	Angelina Jolie	/content/Human-Face-Recognition/dataset/struct...
2	Angelina Jolie	/content/Human-Face-Recognition/dataset/struct...
3	Angelina Jolie	/content/Human-Face-Recognition/dataset/struct...
4	Angelina Jolie	/content/Human-Face-Recognition/dataset/struct...

```
# Cell 5: EDA
plot_distribution(df)
show_samples(df)
```






```
# Cell 6: Train, validation, test split
train_df, test_df = train_test_split(df, test_size=CFG.test_size, random_state=SEED, stratify=df['label'])
val_ratio_adjusted = CFG.val_size / (1 - CFG.test_size)
train_df, val_df = train_test_split(train_df, test_size=val_ratio_adjusted, random_state=SEED, stratify=train_df['label'])
print('Train:', train_df.shape)
print('Validation:', val_df.shape)
print('Test:', test_df.shape)
```

```
Train: (1260, 2)
Validation: (270, 2)
Test: (270, 2)
```

```
# Cell 7: Face detection, alignment, and embedding functions
def align_face(image_bgr, box, keypoints, margin=12):
    x, y, w, h = box
    x = max(0, x - margin)
    y = max(0, y - margin)
    x2 = min(image_bgr.shape[1], x + w + 2 * margin)
    y2 = min(image_bgr.shape[0], y + h + 2 * margin)
    left_eye = keypoints.get('left_eye')
    right_eye = keypoints.get('right_eye')
    if left_eye and right_eye:
        eye_center = ((left_eye[0] + right_eye[0]) / 2, (left_eye[1] + right_eye[1]) / 2)
        angle = np.degrees(np.arctan2(right_eye[1] - left_eye[1], right_eye[0] - left_eye[0]))
        rotation_matrix = cv2.getRotationMatrix2D(eye_center, angle, 1.0)
        image_bgr = cv2.warpAffine(image_bgr, rotation_matrix, (image_bgr.shape[1], image_bgr.shape[0]), flags=cv2.INTER_CUBIC)
    face = image_bgr[y:y2, x:x2]
    if face.size == 0:
        return None
    face = cv2.cvtColor(face, cv2.COLOR_BGR2RGB)
    face = cv2.resize(face, CFG.image_size)
    return face

def extract_single_face(image_path):
    image_bgr = cv2.imread(image_path)
    if image_bgr is None:
        return None, 'image_read_error'
    image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    try:
        detections = detector.detect_faces(image_rgb)
    except ValueError as e:
        # Catch the ValueError specifically from the MTCNN Conv2D if it occurs
        return None, f'mtcnn_detection_error: {e}'
    if not detections:
        return None, 'no_face_detected'
    best = max(detections, key=lambda d: d['box'][2] * d['box'][3])
    aligned = align_face(image_bgr, best['box'], best['keypoints'])
    if aligned is None:
        return None, 'alignment_failed'
    return aligned, None

def build_embeddings(split_df, split_name):
    embeddings, labels, faces, failures = [], [], [], []
    for _, row in tqdm(split_df.iterrows(), total=len(split_df), desc=split_name):
        face, error = extract_single_face(row['image_path'])
        if face is None:
            failures.append({'image_path': row['image_path'], 'label': row['label'], 'error': error})
            continue
        try:
            embedding = embedder.embeddings([face])[0]
            embeddings.append(embedding)
            labels.append(row['label'])
            faces.append(face)
        except Exception as exc:
            failures.append({'image_path': row['image_path'], 'label': row['label'], 'error': str(exc)})
    return {
        'X': np.array(embeddings),
        'y': np.array(labels),
        'faces': np.array(faces),
        'failures': pd.DataFrame(failures)
    }
```

```
train_data = build_embeddings(train_df, 'train')
val_data = build_embeddings(val_df, 'validation')
test_data = build_embeddings(test_df, 'test')

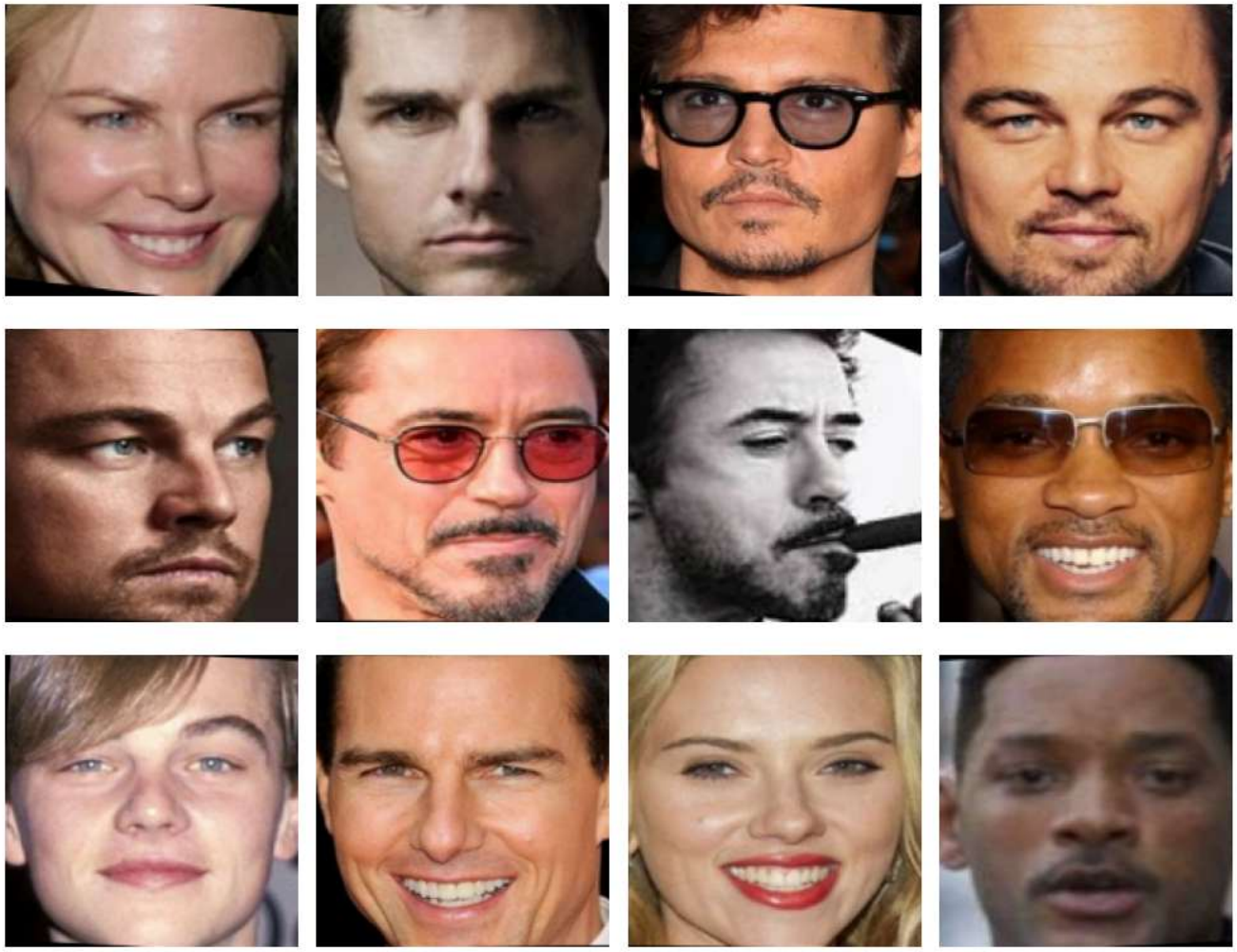
print('Train embeddings:', train_data['X'].shape)
print('Validation embeddings:', val_data['X'].shape)
print('Test embeddings:', test_data['X'].shape)
print('Total failures:', len(train_data['failures']) + len(val_data['failures']) + len(test_data['failures']))

if not train_data['failures'].empty:
    print('\nTrain failures:')
    display(train_data['failures'].head())
```

```
if not val_data['failures'].empty:
    print('\nValidation failures:')
    display(val_data['failures'].head())
if not test_data['failures'].empty:
    print('\nTest failures:')
    display(test_data['failures'].head())
```

Show hidden output

```
# Cell 9: Visualize aligned faces
sample_faces = train_data['faces'][:12]
fig, axes = plt.subplots(3, 4, figsize=(10, 8))
for ax, face in zip(axes.flat, sample_faces):
    ax.imshow(face)
    ax.axis('off')
plt.tight_layout()
plt.show()
```



```
# Cell 10: Encode labels, scale embeddings, define and train classifier
label_encoder = LabelEncoder()
y_train = label_encoder.fit_transform(train_data['y'])
y_val = label_encoder.transform(val_data['y'])
y_test = label_encoder.transform(test_data['y'])

scaler = StandardScaler()
X_train = scaler.fit_transform(train_data['X'])
X_val = scaler.transform(val_data['X'])
X_test = scaler.transform(test_data['X'])

num_classes = len(label_encoder.classes_)
classifier = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.35),
```

```
    Dense(128, activation='relu'),
    Dropout(0.25),
    Dense(num_classes, activation='softmax')
])
classifier.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

callbacks = [
    ModelCheckpoint(MODELS_DIR / 'best_face_classifier.keras', monitor='val_accuracy', save_best_only=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=4, verbose=1),
    EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True, verbose=1)
]

history = classifier.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=CFG.epochs,
    batch_size=CFG.batch_size,
    callbacks=callbacks,
    verbose=1
)

joblib.dump(label_encoder, MODELS_DIR / 'label_encoder.pkl')
joblib.dump(scaler, MODELS_DIR / 'scaler.pkl')
print('Model and preprocessing artifacts saved.')
```

```
Epoch 1/10
34/39 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5565 - loss: 1.6848
Epoch 1: val_accuracy improved from None to 0.98885, saving model to /content/Human-Face-Recognition/models/best_face_classifier

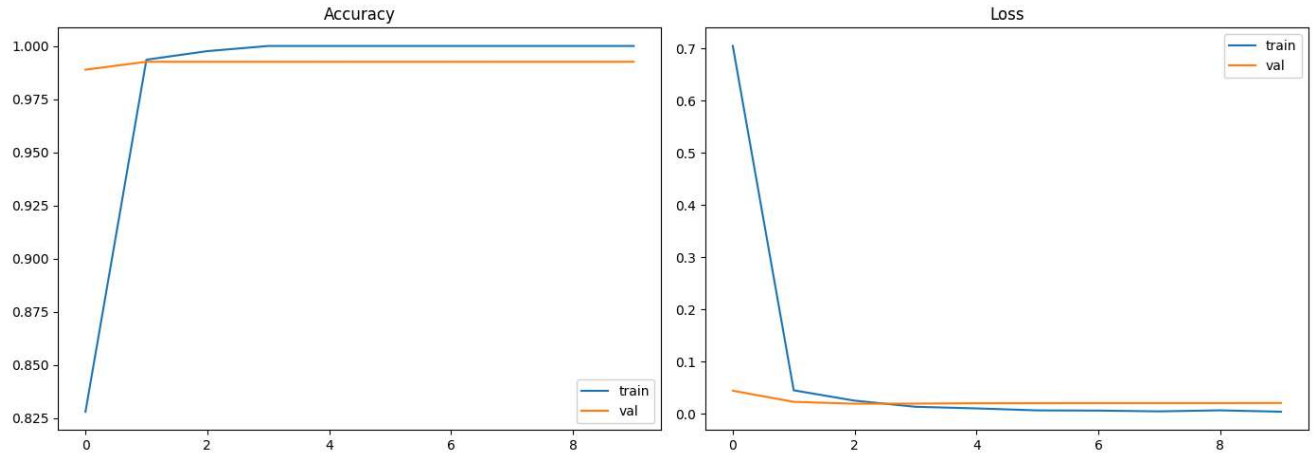
Epoch 1: finished saving model to /content/Human-Face-Recognition/models/best_face_classifier.keras
39/39 ━━━━━━━━━━━━━━━━━━━━ 4s 29ms/step - accuracy: 0.8279 - loss: 0.7046 - val_accuracy: 0.9888 - val_loss: 0.0441 - learning_r
Epoch 2/10
34/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9946 - loss: 0.0522
Epoch 2: val_accuracy improved from 0.98885 to 0.99257, saving model to /content/Human-Face-Recognition/models/best_face_classif

Epoch 2: finished saving model to /content/Human-Face-Recognition/models/best_face_classifier.keras
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9935 - loss: 0.0449 - val_accuracy: 0.9926 - val_loss: 0.0228 - learning_ra
Epoch 3/10
30/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9975 - loss: 0.0340
Epoch 3: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9976 - loss: 0.0251 - val_accuracy: 0.9926 - val_loss: 0.0192 - learning_ra
Epoch 4/10
38/39 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0151
Epoch 4: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0133 - val_accuracy: 0.9926 - val_loss: 0.0194 - learning_ra
Epoch 5/10
32/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0121
Epoch 5: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0102 - val_accuracy: 0.9926 - val_loss: 0.0201 - learning_ra
Epoch 6/10
34/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0063
Epoch 6: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 1.0000 - loss: 0.0064 - val_accuracy: 0.9926 - val_loss: 0.0203 - learning_ra
Epoch 7/10
32/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0059
Epoch 7: val_accuracy did not improve from 0.99257

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0060 - val_accuracy: 0.9926 - val_loss: 0.0204 - learning_ra
Epoch 8/10
32/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0056
Epoch 8: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0045 - val_accuracy: 0.9926 - val_loss: 0.0204 - learning_ra
Epoch 9/10
32/39 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0063
Epoch 9: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0065 - val_accuracy: 0.9926 - val_loss: 0.0204 - learning_ra
Epoch 10/10
39/39 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0037
Epoch 10: val_accuracy did not improve from 0.99257
39/39 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 1.0000 - loss: 0.0039 - val_accuracy: 0.9926 - val_loss: 0.0206 - learning_r
Restoring model weights from the end of the best epoch: 3.
Model and preprocessing artifacts saved.
```

```
# Cell 11: Training graphs
history_df = pd.DataFrame(history.history)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(history_df['accuracy'], label='train')
axes[0].plot(history_df['val_accuracy'], label='val')
axes[0].set_title('Accuracy')
```

```
axes[0].legend()
axes[1].plot(history_df['loss'], label='train')
axes[1].plot(history_df['val_loss'], label='val')
axes[1].set_title('Loss')
axes[1].legend()
plt.tight_layout()
plt.show()
```




```
# Cell 12: Evaluation and confusion matrix
best_model = load_model(MODELS_DIR / 'best_face_classifier.keras')
y_prob = best_model.predict(X_test, verbose=0)
y_pred = np.argmax(y_prob, axis=1)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print(f'Accuracy : {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall   : {recall:.4f}')
print(f'F1-score : {f1:.4f}')
print('\nClassification report:\n')
print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

class_centroids = []
for class_name in label_encoder.classes_:
    class_embeddings = train_data['X'][train_data['y'] == class_name]
    class_centroids.append(class_embeddings.mean(axis=0))
class_centroids = np.array(class_centroids)
np.save(MODELS_DIR / 'class_centroids.npy', class_centroids)
np.save(MODELS_DIR / 'class_names.npy', label_encoder.classes_)
```

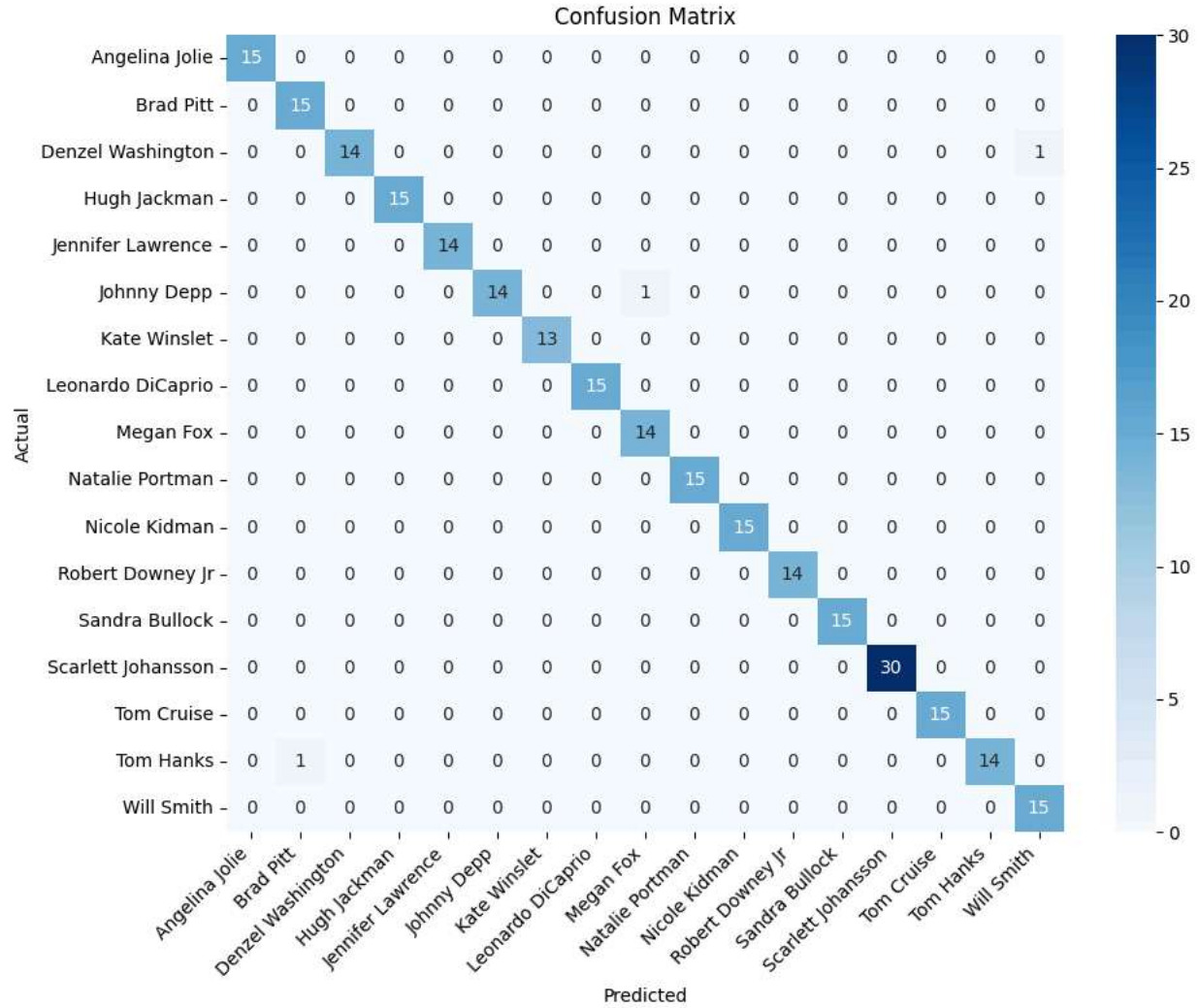
```
Accuracy : 0.9887
Precision: 0.9894
Recall   : 0.9887
F1-score : 0.9887

Classification report:

                   precision    recall  f1-score   support

    Angelina Jolie       1.00      1.00      1.00        15
         Brad Pitt       0.94      1.00      0.97        15
  Denzel Washington      1.00      0.93      0.97        15
       Hugh Jackman      1.00      1.00      1.00        15
  Jennifer Lawrence      1.00      1.00      1.00        14
        Johnny Depp      1.00      0.93      0.97        15
       Kate Winslet      1.00      1.00      1.00        13
  Leonardo DiCaprio      1.00      1.00      1.00        15
         Megan Fox       0.93      1.00      0.97        14
    Natalie Portman      1.00      1.00      1.00        15
      Nicole Kidman      1.00      1.00      1.00        15
   Robert Downey Jr      1.00      1.00      1.00        14
     Sandra Bullock      1.00      1.00      1.00        15
 Scarlett Johansson      1.00      1.00      1.00        30
         Tom Cruise      1.00      1.00      1.00        15
          Tom Hanks      1.00      0.93      0.97        15
         Will Smith      0.94      1.00      0.97        15

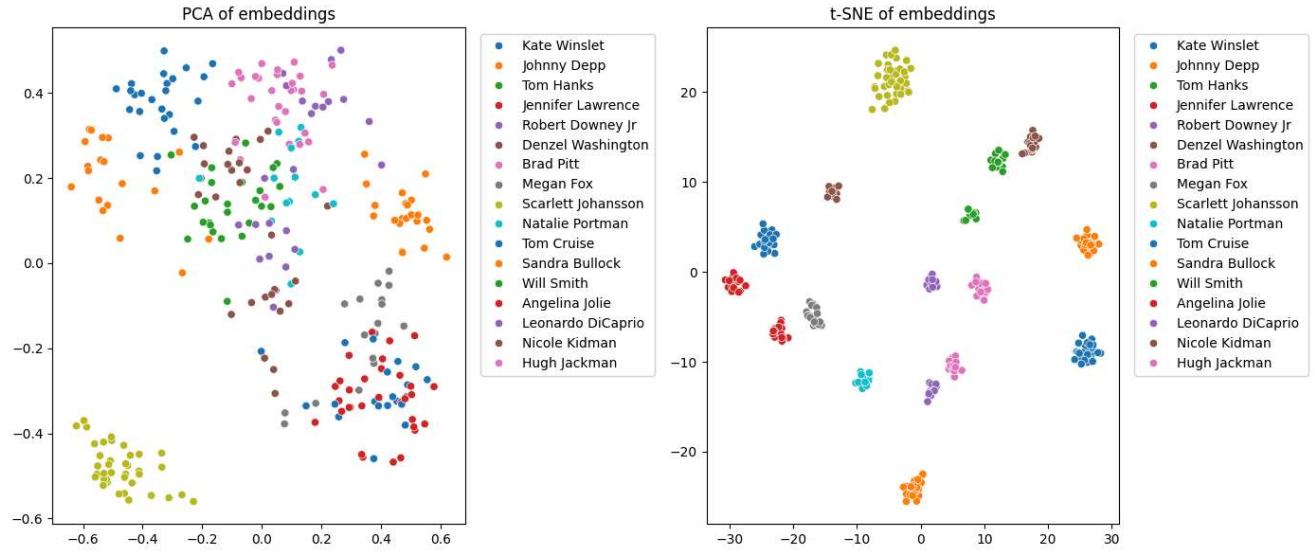
           accuracy                           0.99       265
          macro avg       0.99      0.99      0.99       265
       weighted avg       0.99      0.99      0.99       265
```




```
# Cell 13: PCA and t-SNE visualization of FaceNet embeddings
sample_n = min(300, len(train_data['X']))
idx = np.random.choice(len(train_data['X']), sample_n, replace=False)
sample_X = train_data['X'][idx]
sample_y = train_data['y'][idx]
```

```
pca = PCA(n_components=2, random_state=SEED)
pca_2d = pca.fit_transform(sample_X)
tsne = TSNE(n_components=2, random_state=SEED, perplexity=20, init='random', learning_rate='auto')
tsne_2d = tsne.fit_transform(sample_X)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
sns.scatterplot(x=pca_2d[:, 0], y=pca_2d[:, 1], hue=sample_y, ax=axes[0], palette='tab10')
axes[0].set_title('PCA of embeddings')
sns.scatterplot(x=tsne_2d[:, 0], y=tsne_2d[:, 1], hue=sample_y, ax=axes[1], palette='tab10')
axes[1].set_title('t-SNE of embeddings')
axes[0].legend(bbox_to_anchor=(1.02, 1), loc='upper left')
axes[1].legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```




```
# Cell 14: Inference helpers for uploaded image, image URL, unknown detection, and attendance
ATTENDANCE_CSV = MODELS_DIR / 'attendance.csv'

def mark_attendance(name):
    if name == 'Unknown':
        return
    timestamp = pd.Timestamp.now()
    row = {'name': name, 'date': timestamp.date().isoformat(), 'time': timestamp.strftime('%H:%M:%S')}
    if ATTENDANCE_CSV.exists():
        df_att = pd.read_csv(ATTENDANCE_CSV)
        already = ((df_att['name'] == row['name']) & (df_att['date'] == row['date'])).any()
        if already:
            return
        df_att = pd.concat([df_att, pd.DataFrame([row])], ignore_index=True)
    else:
        df_att = pd.DataFrame([row])
    df_att.to_csv(ATTENDANCE_CSV, index=False)

def predict_faces_from_rgb(image_rgb):
    image_bgr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)
    detections = detector.detect_faces(image_rgb)
    results = []
    annotated = image_rgb.copy()
    for det in detections:
        aligned = align_face(image_bgr, det['box'], det['keypoints'])
        if aligned is None:
            continue
        embedding = embedder.embeddings([aligned])[0]
        scaled = scaler.transform([embedding])
        probs = best_model.predict(scaled, verbose=0)[0]
        pred_idx = int(np.argmax(probs))
        pred_name = label_encoder.inverse_transform([pred_idx])[0]
        pred_conf = float(probs[pred_idx])
        centroid_idx = int(np.where(label_encoder.classes_ == pred_name)[0][0])
```

```
            distance = float(cosine_distances([embedding], [class_centroids[centroid_idx]])[0][0])
            if distance > CFG.unknown_distance_threshold or pred_conf < CFG.classifier_confidence_threshold:
                final_name = 'Unknown'
                final_conf = max(0.0, 1.0 - distance)
            else:
                final_name = pred_name
                final_conf = pred_conf
                mark_attendance(final_name)
            x, y, w, h = det['box']
            color = (0, 255, 0) if final_name != 'Unknown' else (255, 60, 60)
            cv2.rectangle(annotated, (x, y), (x + w, y + h), color, 2)
            cv2.putText(annotated, f'{final_name} | {final_conf:.2f}', (x, max(25, y - 10)), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color,
            results.append({'identity': final_name, 'confidence': final_conf, 'distance': distance, 'box': det['box']})
    return annotated, pd.DataFrame(results)

def load_image_from_url(url):
    response = requests.get(url, timeout=30)
    response.raise_for_status()
    arr = np.frombuffer(response.content, dtype=np.uint8)
    image_bgr = cv2.imdecode(arr, cv2.IMREAD_COLOR)
    if image_bgr is None:
        raise ValueError('Could not decode image from URL.')
    return cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
```

```
# Cell 15: Test on uploaded custom image
from google.colab import files
uploaded = files.upload()
if uploaded:
    file_name = next(iter(uploaded))
    file_bytes = np.frombuffer(uploaded[file_name], np.uint8)
    image_bgr = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)
    image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    annotated, result_df = predict_faces_from_rgb(image_rgb)
    plt.figure(figsize=(10, 8))
    plt.imshow(annotated)
    plt.axis('off')
    plt.show()
    display(result_df)
```

Choose Files download.jpeg

download.jpeg(image/jpeg) - 3771 bytes, last modified: 4/26/2026 - 100% done

```
Saving download.jpeg to download (4).jpeg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step
```

